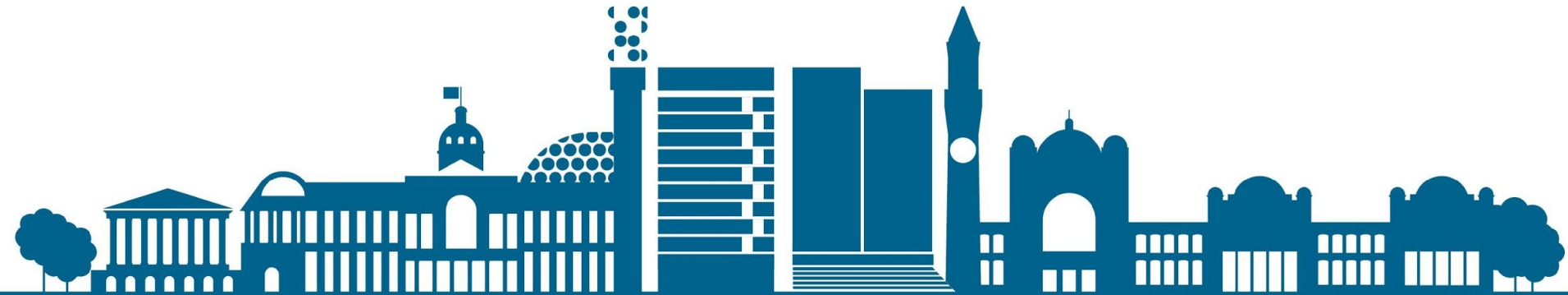




UNIVERSITY OF
BIRMINGHAM

Functional Programming Introduction & Game of Life

Eric Finster
Martin Escardo
Mian M. Hamayun



Module Outline

◆ Week 1

- Introduction to Haskell - Conway's Game of Life.
- Types in Haskell
- Polymorphism

◆ Week 2

- Type classes and instances
- Functions
- More on type classes
- Even more on type classes

◆ Week 3

- List comprehensions
- Recursive functions
- Higher-order functions



Module Outline (cont'd)

- ◆ Week 4 & 5
 - User defined data types I
 - Lazy natural numbers
- ◆ Week 6
 - Consolidation Week
- ◆ Week 7
 - User defined data types II
- ◆ Week 8 & 9
 - Monads
- ◆ Week 10
 - Implementing a small imperative language
- ◆ Week 11
 - Implementing a small imperative language
 - Rigorous programming
- ◆ Additional, optional materials
 - A model IO' of the IO monad
 - Unbeatable tic-tac-toe
 - Elements of Laziness
- ◆ [Module Outline on Gitlab](#)



Module Learning Outcomes

On successful completion of this module, students should be able to:

- 1) Present the basic ideas of functional programming languages
- 2) Demonstrate the main elements of good functional programming style
- 3) Illustrate some of the uses and applications of functional programming
- 4) Understand and apply more advanced features of typed functional programming.



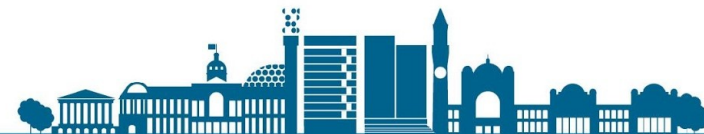
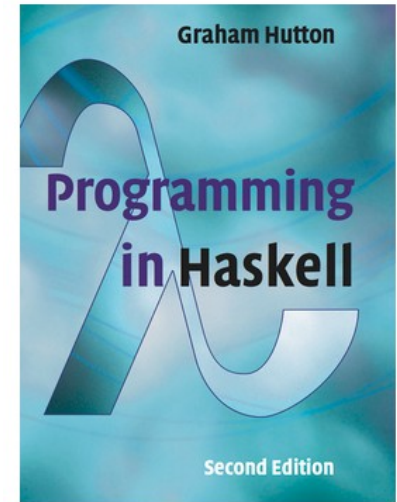
Important General Information

- ◆ All module files are kept on [GitLab](#) rather than on Canvas. This includes:
 - Lecture Notes
 - Weekly Activity Plans
 - Problem Sheets
 - Assessed Assignments (Class Tests)
- ◆ and other relevant stuff including
 - Notes on Plagiarism
 - Link to [Hoogle](#) (for finding library functions)
 - Canvas page
 - Link to [Teams Student Support](#)



Module Textbook & Assessments

- ◆ Officially adopted textbook:
 - **Programming in Haskell**
 - Graham Hutton
 - Cambridge University Press
 - Paperback: ISBN 978-1316626221; Kindle: ASIN B01JGMEA3U
 - 318 pages, 120 exercises
- ◆ Other free books include
 - [Learn you a Haskell for Great Good](#)
 - [A Gentle Introduction to Haskell](#)
 - [Real World Haskell](#)



Assessment Plan & Organization

Lectures / Labs (Edgbaston)

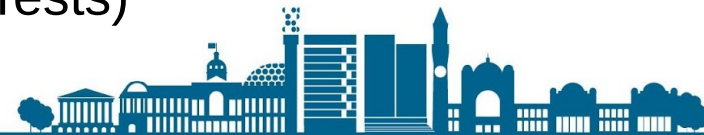
- ◆ Lectures: Wednesdays, 9:00am-11:00am (UK)
- ◆ Labs
 - Thursdays, 2pm-3pm & 3pm-4pm, UG04 Computer Science building
 - Fridays, 2pm-3pm & Fri 3pm-4pm, UG04 Computer Science building

Lectures / Labs (Dubai)

- ◆ Lectures: Tuesdays, 11:00am-1:00pm (UAE)
- ◆ Labs
 - Thursdays, 10am-12pm (UAE), 0411 – IT Cluster (Sec. A)
 - Thursdays, 1pm-3pm (UAE), 0411 – IT Cluster (Sec. B)

Assessments (more details given [Here](#))

- ◆ 100% Continuous Assessments (2 Class Tests)



Motivational Example

- ◆ In an imperative programming language, how can we compute the sum from 0 to a given number?

```
int total = 0;
for (int count = 1; count <= n; count++)
    total = total + count;
```

- ◆ How about Haskell?

```
> sum [1..n]
```

```
sum :: Num a => [a] -> a
```

```
sum [] = 0
```

```
sum (x:xs) = x + sum xs
```



The Zoo of Functional Programming Languages

◆ **Untyped** Programming Languages

- **LISP** 1958 (**L**ots of **I**nsanely **S**tupid **P**arentheses - old joke :)
 - Untyped (rather only one type ie. S-expression)
- Many languages in LISP family e.g. scheme, emacs lisp, ...

◆ **Typed** Programming Languages (infact strongly typed)

- **ML** (Meta Language for theorem proving, developed at university of Edinburgh)
- **SML** (1983), **OCAML** (1996), **F#** (2005, Microsoft)
- **Hope** (1970), **Miranda** (1985), **Haskell** (1990)
- **Coq**, **Agda**, **Lean** (dependently typed, concrete proofs)
 - write proofs of computer programs / correctness.



Other Languages with Functional Features

◆ **Java**

- lambda (anonymous functions)
- functional interfaces
- streams

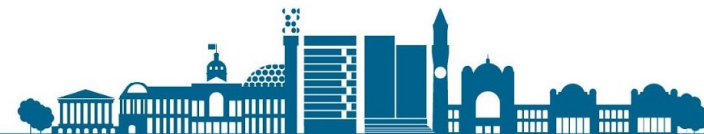
◆ **Kotlin**

- The preferred language for Android development since 2019
- More heavily based on functional programming ideas.



What is Functional Programming?

- ◆ A function is a mapping that takes one or more arguments and produces a single result.
- ◆ Functional programming is a **style of programming**, where the basic method of computation is the application of functions to arguments.
- ◆ A functional programming language
 - allows storage of functions in data structures
 - constructing intermediate structures
 - taking functions as arguments
 - producing functions as results
 - functions defined in terms of themselves.

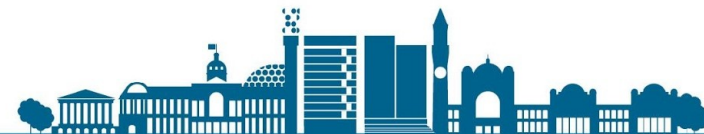


Learning Objective

To discuss a **self-contained Haskell program** that has many features that we will study in the next few lectures.

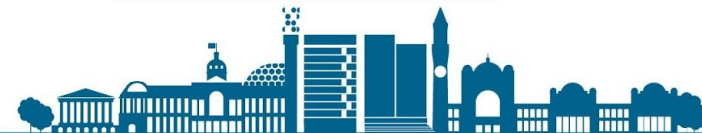
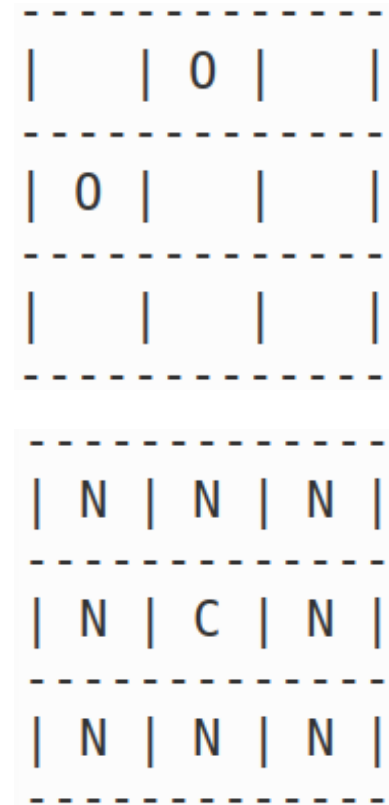
Don't worry if you don't understand everything.

Our objective is to **motivate** the usefulness of these features with an **entertaining example**.



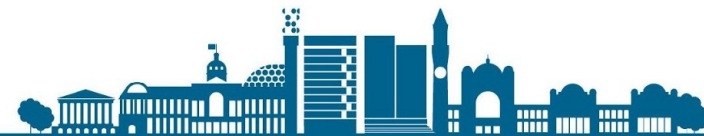
Conway's Game of Life

- ◆ The [Game of Life](#), invented by [John Conway](#), was an early example of a cellular automaton with complex behaviors generated from a simple set of rules.
- ◆ The Game of Life begins by placing some **tokens** on the cells of a **two-dimensional grid (infinite)**.
 - Cells of the grid that contain a token are said to be **live**, while empty cells are said to be **dead**.
 - Each cell has exactly **eight neighbours**, corresponding to the cells that are immediately adjacent horizontally, vertically, or diagonally.
 - The initial configuration is called the **seed**, or **generation 0**.



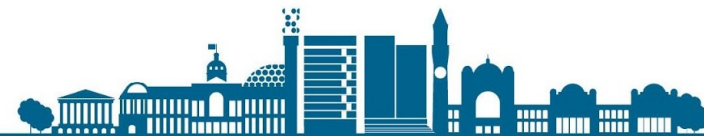
Conway's Game of Life – Rules

- ◆ The game is played deterministically according to the following rules of evolution whereby a cell interacts with its neighbours:
 - **[Birth.]** A dead cell with **exactly three live neighbours** becomes live at the next generation.
 - **[Survival.]** A live cell with **either two or three live neighbours** survives to the next generation.
 - **[Death.]** A cell dies from overcrowding if it has **more than three live neighbours**, or from isolation if it has **fewer than two neighbours**.
- ◆ It is important to clarify that the rules specify that **all of these births and deaths happen simultaneously** as the grid evolves into the next generation, rather than in any particular order.



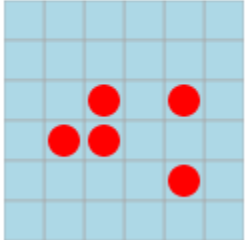
Conway's Game of Life – Rules (Simplified)

- ◆ The game proceeds in discrete steps of time and following the evolution function / rules given below:
 1. Any live cell with **0 or 1 live neighbours dies** by underpopulation.
 2. Any live cell with **2 or 3 live neighbours survives**.
 3. Any live cell with **more than 3 live neighbours dies** by overpopulation.
 4. Any dead cell with **3 live neighbours comes to life** by reproduction.
- ◆ **Important:** You have to apply these rules on the grid at the **SAME TIME for each cell**; If you apply these rules in any chosen order, its not going to work!

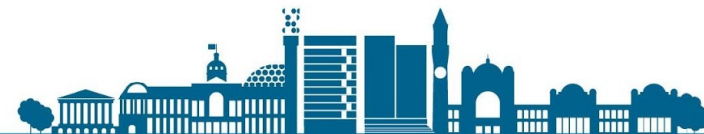
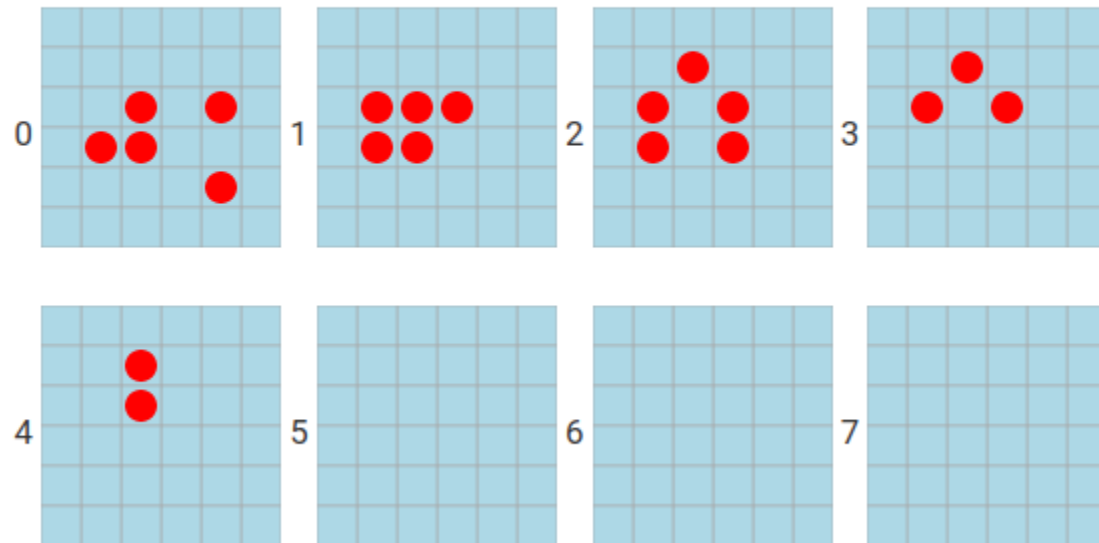


Conway's Game of Life - Examples

Many initial configurations quickly die off. For example, the seed

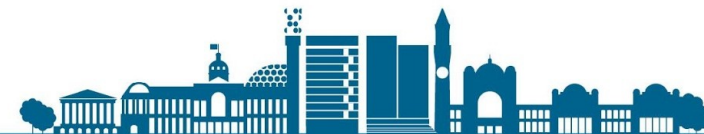
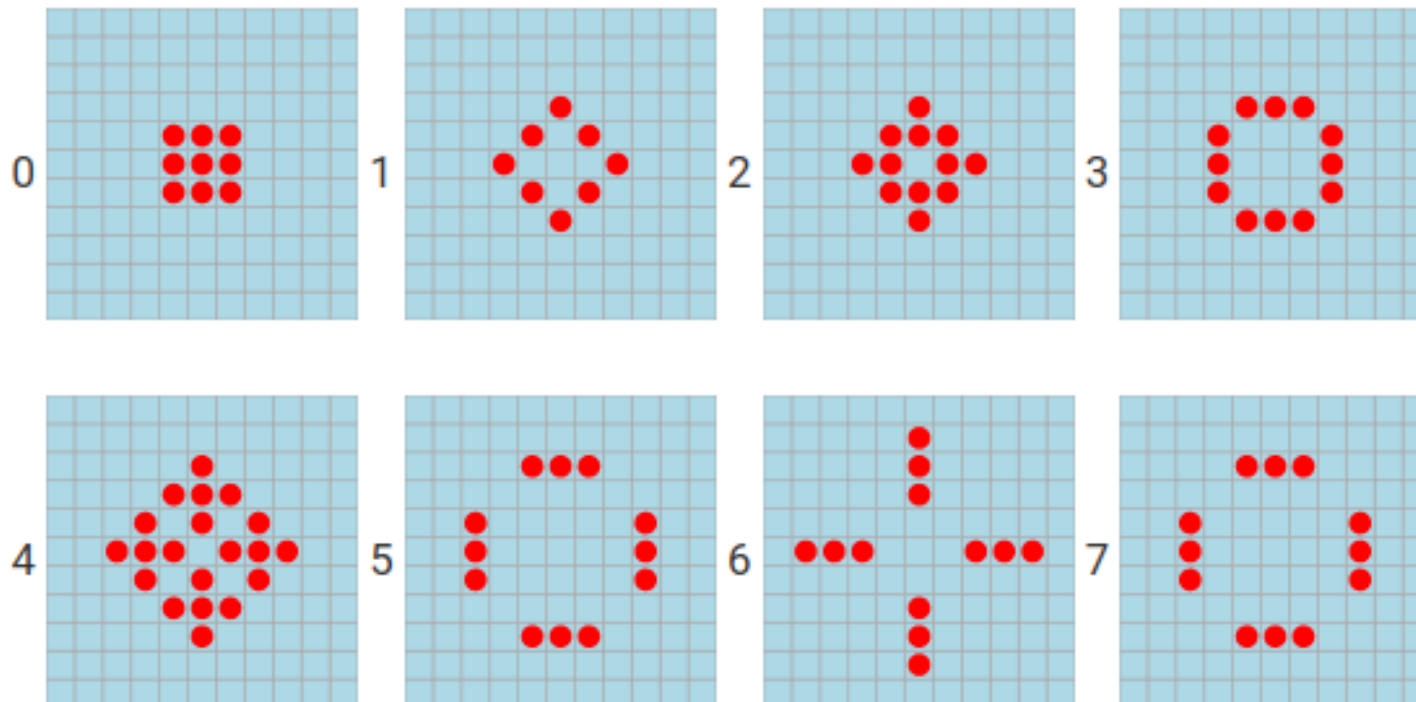


evolves to an empty grid after five generations. (Since an empty grid can never be repopulated, all generations after generation 5 are the same.)

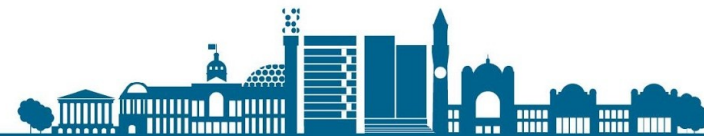
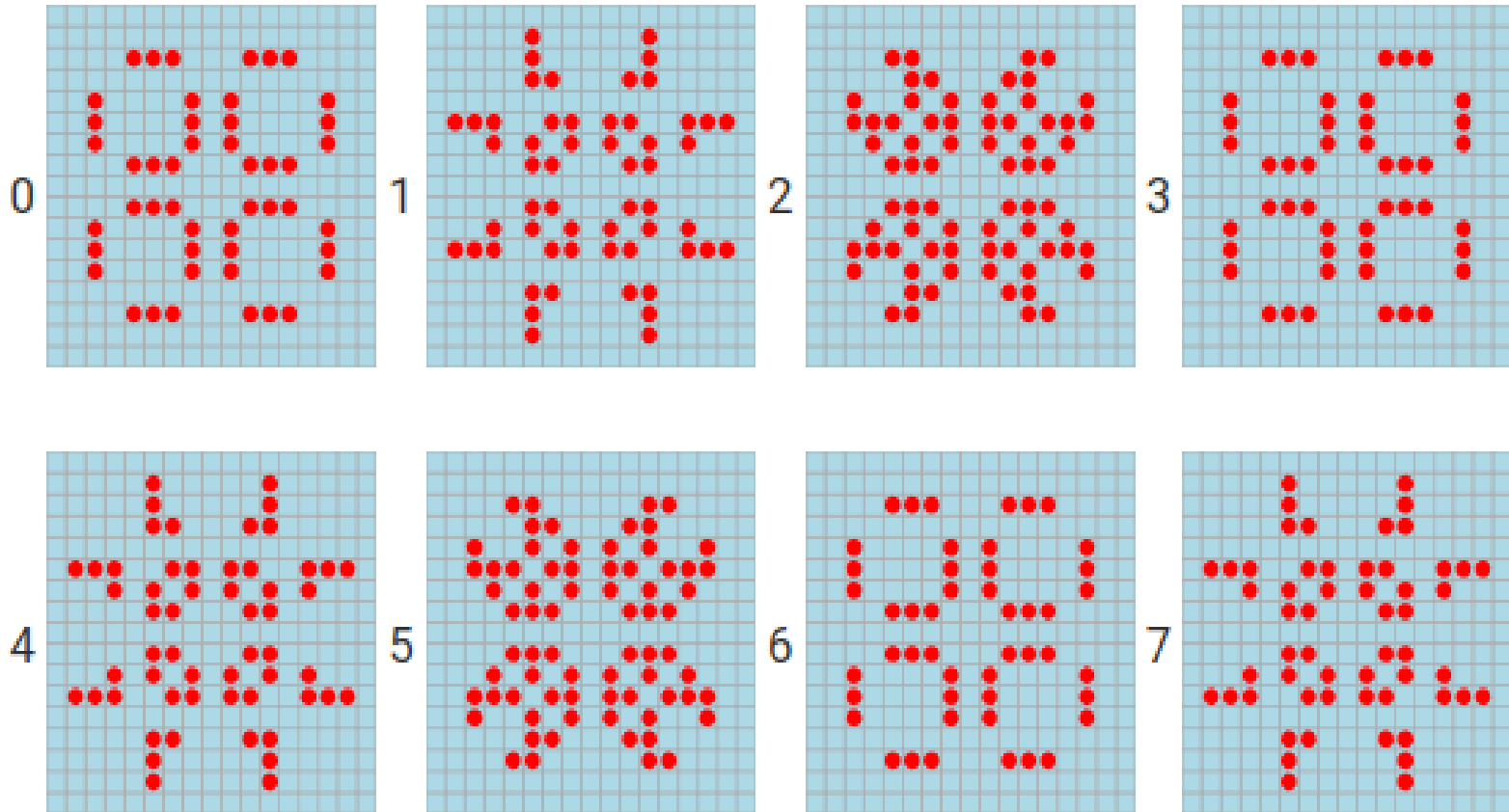


Conway's Game of Life - Examples

On the other hand, some seeds enter into a stable pattern of oscillation between a collection of two or more states.

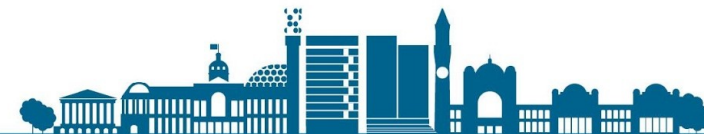
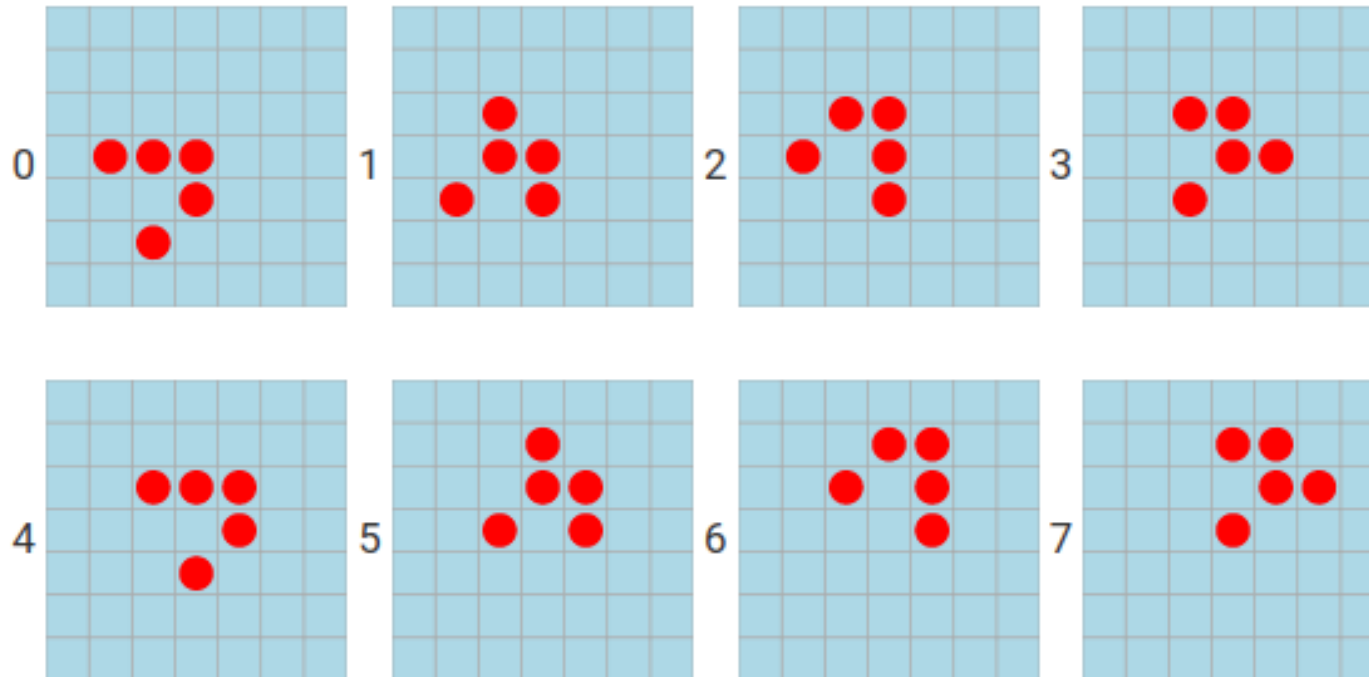


Conway's Game of Life - Examples



Conway's Game of Life - Examples

Another interesting example is Conway's *glider* pattern, which moves slowly across the board, drifting up and to the right by one square every four generations.



Game of Life – Implementation in Haskell

We make the following type declarations:

```
type Cell = (Int,Int)
```

```
type Grid = [Cell]
```

This says that a cell is represented by a pair of integers.

A grid is represented by a list of cells. (Its xy coordinates)

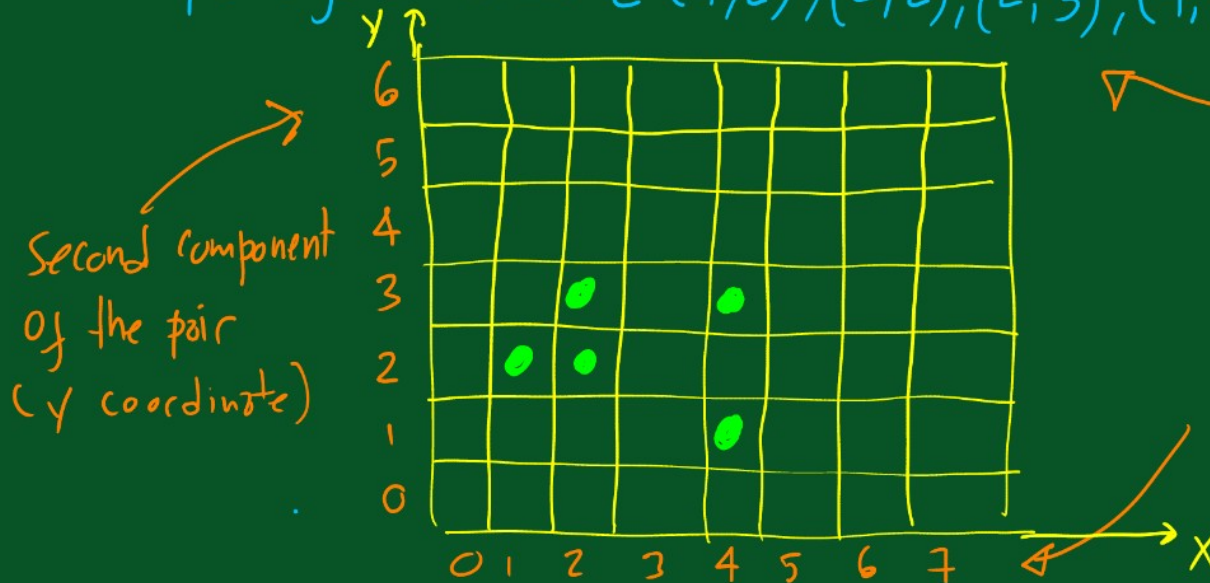


Game of Life – Implementation in Haskell (2)

Example: The pentagenzion starting pattern.

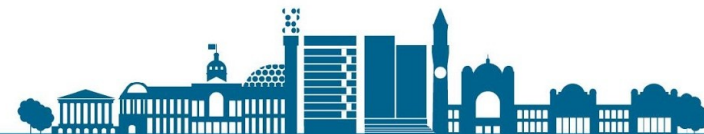
pentagenzion :: Grid $\leftarrow$ type declaration.

pentagenzion = [(1,2),(2,2),(2,3),(4,1),(4,3)] $\leftarrow$ definition.



∇ the order of the pairs in the list doesn't matter.

first component of the pair (x coordinate)



Game of Life – Implementation in Haskell (3)

Checking whether a cell is live or dead:

$\text{isLive, isDead} :: \text{Cell} \rightarrow \text{Grid} \rightarrow \text{Bool}$ two type declarations
separated by comma

$\text{isLive } c \ g = c \text{ `elem` } g$

$\text{isDead } c \ g = \text{not}(\text{isLive } c \ g)$ definitions of the
two declared
functions

The two functions isLive and isDead take two inputs (namely a cell c and a grid g) and return a boolean True or False .

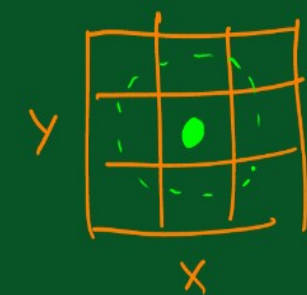


Game of Life – Implementation in Haskell (4)

The list of neighbours of a cell

$\text{neighbours} :: \text{Cell} \rightarrow [\text{Cell}]$

definition
by "pattern
matching"



$\text{neighbours } (x,y) = [(x+i, y+j) \mid i \leftarrow [-1..1],$
 $j \leftarrow [-1..1],$
 $\text{not } (i==0 \ \&\& \ j==0)]$

"list comprehension"

This is the list of all $(x+i, y+j)$ where i and j range from -1 to 1 , excluding the case in which both are zero.



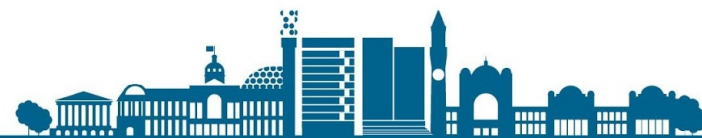
Game of Life – Implementation in Haskell (5)

The live neighbours of a cell in a grid

`liveNeighbours :: Grid → Cell → [Cell]`

`liveNeighbours g c = [ c' | c' ← neighbours c, isLive c' g ]`

↳ list comprehension again



Game of Life – Implementation in Haskell (5)

The evolution function *step* of the game of life

$\text{step} :: \text{Grid} \rightarrow \text{Grid}$

$\text{step } g = [(x,y) \mid x \leftarrow [\text{minX}-1 .. \text{maxX}+1],$
 $y \leftarrow [\text{minY}-1 .. \text{maxY}+1],$

$(\text{isDead } (x,y) \text{ } g \ \&\& \ \text{length } (\text{fiveNeighbours } g (x,y)) == 3)$
 $\parallel (\text{isLive } (x,y) \text{ } g \ \&\& \ \text{length } (\text{liveNeighbours } g (x,y)) \notin [2,3])$

where

$\text{minX} = \text{minimum } [x \mid (x,y) \leftarrow g]$

$\text{maxX} = \text{maximum } [x \mid (x,y) \leftarrow g]$

$\text{minY} = \text{minimum } [y \mid (x,y) \leftarrow g]$

$\text{maxY} = \text{maximum } [y \mid (x,y) \leftarrow g]$



Game of Life – Implementation in Haskell (6)

`terminalWidth = 70` | $\leftarrow$ We don't need to declare the types.
`terminalHeight = 22` | They are inferred automatically.

The terminal accepts control commands as special characters.

Clear the terminal:

`cls :: IO ()` $\leftarrow$ type of programs that perform
input/output actions and return "nothing".

`cls = putStr "\ESC[2J"`

$\uparrow$ Terminal control sequence (see table)
(ANSI escape code)



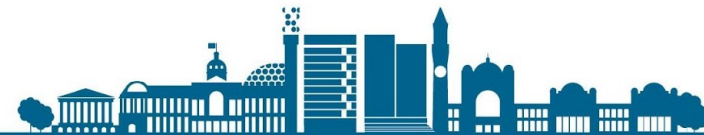
Game of Life – Implementation in Haskell (7)

Move cursor to a given position, or cell

```
goto :: Cell → IO()
```

```
goto (x,y) = putStr "\ESC["  
            ++ show (termiz/height - y)  
            ++ ";"  
            ++ show (x+1)  
            ++ "H")
```

corresponds to Java's
toString()



Game of Life – Implementation in Haskell (8)

Printing a cell in the terminal

```
printCell :: Cell → IO ()
```

```
printCell (x,y) | x ≥ 0 && x < terminalWidth  
                && y ≥ 0 && y < terminalHeight = do  
    goto (x,y)  
    putChar 'O'
```

```
| otherwise = return ()
```

Performs no I/O action and returns the value ().

prints the letter O
and returns the
value ()

goto (x,y)
putChar 'O'



Game of Life – Implementation in Haskell (9)

Render a grid in the terminal

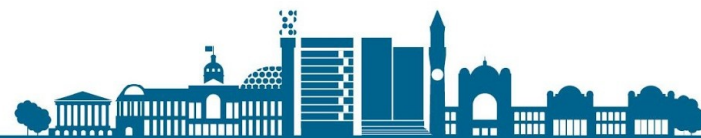
terminalRender :: Grid → IO()

terminalRender g = do
 cls

List of I/O actions,
one for each cell c
in g .

→ sequence [printCell c | c ← g]
goto (0, terminalHeight)

Performs a list of I/O actions sequentially, one after the other.



Game of Life – Implementation in Haskell (10)

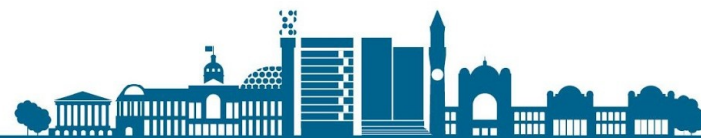
10^{th} of a second delay

`delayTenthSec n = threadDelay (n *  $10^5$ )`

Library function from the import `Control.Concurrent`.

Whoops! we forgot to declare the type of this function.

- It is automatically inferred to be `Int → IO ()`.
- But I strongly recommend to add types, for both debugging and documentation. It makes programs more readable.



Game of Life – Implementation in Haskell (11)

Finally, we animate the game in the terminal

$\text{life} :: \text{Grid} \rightarrow \text{IO}()$

$\text{life seed} = f 0 \text{ seed}$

where

$f\ n\ g = \text{do}$ terminalRender g
putStrLn (show n)
delayTenthSec 1
 $f\ (n+1)\ (\text{step } g)$



Game of Life – Practice

- ◆ Run Game of Life using `runhaskell` command or `ghci`, using interactive mode.
- ◆ Try out different seed grids; increase decrease delay.
- ◆ Compile using `ghc` (`ghc --make Life.hs`) and then try it out.
 - https://downloads.haskell.org/~ghc/latest/docs/html/users_guide/using.html
- ◆ How to get the runnable haskell code from lecture handouts.
 - `cat Life.md | runhaskell ../../Resources/mdtohs.hs > Life2.hs`



Summary

- ◆ We have also developed a version in [Kotlin](#) for you.
 - Check the resources for the module for both the [Haskell](#) and the [Kotlin](#) files. Read the lecture notes too.
 - **Can you translate them to Java**, while maintaining their functional style?
- ◆ We used the Game of Life to illustrate the features of Haskell that we'll learn in the future lectures.

